Appendix C: IEEE 830 Template

Software Requirements Specification

for

ROM-Scan Analyzer

Version 2

Prepared by Kacper Pawłowski

IFE/WEEIA 4CS3

8.05.26

«Page Break»

Table of Contents

«Table of Contents»

Revision History

Name	Date	Reason for Changes	Version
Initial version	27.04.26	Initial draft	v1
Description and Requirements	8.05.26	Adding UML description diagram and filling out the corresponding part/.	v2

«Page break»

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for the ROM-Scan Analyzer, version 1.0. This product is designed to scan local directories to identify and categorize video game ROM files. The

scope covers the initial development phase, focusing on the directory parsing engine and the interface for displaying ROM metadata.

1.2 Document Conventions

This specification follows the IEEE 830 standard. Bold text is used for UI elements, while italics are used for references. Every requirement statement is assigned a unique identifier (e.g., REQ-1) and a priority level: High, Medium, or Low. Higher-level requirements pass their priority down to detailed requirements.

1.3 Intended Audience and Reading Suggestions

This document is intended for Developers (to guide implementation), Laboratory Instructors (for verification), and Documentation Writers. The document is organized into six sections; it is recommended to read the Product Scope (1.4) and Overall Description (Section 2) first before proceeding to System Features.

1.4 Product Scope

Currently, digital game enthusiasts often possess thousands of ROM files spread across inconsistent directory structures, often with cryptic filenames (e.g., "smb_v1.zip"). This makes library management a manual, time-consuming process.

The ROM-Scan Analyzer addresses this problem by providing an automated parsing engine. The system will bridge the gap between "raw data" and a "user-friendly library" by identifying files based on internal headers and standard naming conventions. While the final application vision includes metadata fetching and emulator integration, this laboratory project focuses on the core challenge: the reliable identification and metadata display of local ROM files. This establishes a testable, foundational module for the larger ecosystem.

1.5 References

1. IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.

2. Course Assignment Document: Laboratory Project Guidelines.

3. ROM Hacking/Preservation Standards: Documentation regarding common ROM file signatures.2.1. Product Perspective

«Describe the context and origin of the product being specified in this specification. For example, state whether this product is a follow-on member of a product family, a replacement for certain existing systems, or a new, self-contained product.»

2.2. Product Features

«Summarize the major features the product must perform or must let the user perform. Details will be provided in Section 3, so only a bullet list is needed here. Organize the features to make them understandable to any reader of the specification. A picture of the major groups of related requirements and how they relate, such as a top level data flow diagram or object class diagram, is often effective.»

- «Thing 1»
- «More things as required»

2.3. User Classes and Characteristics

«Identify the various user classes (intended target for the product) that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.»

2.4. Operating Environment

«Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.»

2.5. Design and Implementation Constraints

«Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).»

2.6. User Documentation

«List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.»

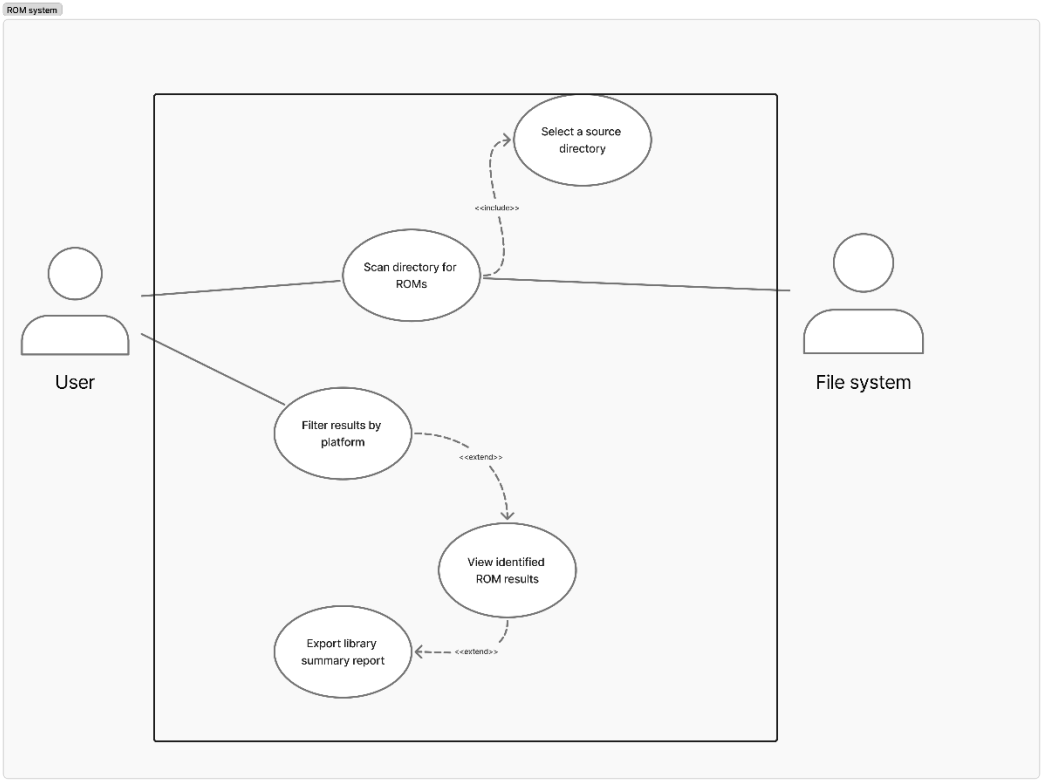
2.7. Assumptions and Dependencies

«List any assumed factors (as opposed to known facts) that could affect the requirements stated in the specification. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).»

3. System Features

«The purpose here is to give a brief introduction to the system features, so that §4 has something to reference. Then, the full explanation of the system features is given in §5, likely referencing §4. The numbering in §3 and

§5 should match.»



3.1. Feature: Select Source Directory

- **Description: The system allows the user to browse local storage media or input an explicit absolute directory path to establish the root workspace for subsequent operations.**
- **Functional Alignment: This feature serves as a mandatory pre-requisite, passing the target path variable directly to the execution loop of the scanning engine.**

3.2. Feature: Scan Directory for ROMs

- **Description: The system recursively traverses the designated file path, parses binary file structures, and validates files against an internal header signature database.**
- **Functional Alignment: This feature encapsulates the primary parsing mechanism of the utility, mapping internal file system attributes onto readable metadata.**

3.3. Feature: View Identified ROM Results

- **Description: The system displays a structured interface populated with the extracted attributes of all successfully validated ROM files.**
- **Functional Alignment: Acts as the primary data presentation layer, serving as the base screen from which users can trigger filtering or reporting sub-systems.**

3.4. Feature: Filter Results by Platform

- **Description: The system enables users to narrow the active display list dynamically using explicit criteria such as target gaming console or file type extensions.**
- **Functional Alignment: An optional, user-initiated extension that manipulates the visibility layer of the active library state without re-running a directory scan.**

3.5. Feature: Export Library Summary Report

- **Description: The system compiles the current active library configuration and writes a structured summary index to an external flat file.**

- **Functional Alignment: An optional reporting engine that outputs independent tracking metrics for historical storage use or external library verification.**

4. External Interface Requirements

4.1. User Interfaces Overview

«Describe the high level functionality of the system from the user's perspective. Explain how the user should be able to use your system to complete all the expected features and the feedback information that will be displayed for the user (for each screen the user will see). For example, specify whether it is GUI or text based interface and how at high level it would work. Discuss at high level how user will interact with the game such as clicking buttons, typing in some selection character, etc.»

«Describe the minimum requirements for the interface. This may include some sample screen images whether for text or GUI approach, any GUI standards or product family style guides that are to be followed, screen layout constraints, for GUI standard buttons and functions (e.g., help) that must appear on every screen, keyboard shortcuts, error message display standards, and so on»

4.2. Hardware Interfaces

«Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.»

4.3. Software Interfaces

«Describe the connections between this product and other specific software components (name and version), including databases, interpreters, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.»

5. System Features/Modules

«This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product.»

5.1. **Scan Directory for ROMs**

5.1.1 Description and Priority

The system parses a user-specified local folder structure to identify valid game files based on internal signature headers. **Priority: High.**

5.1.2 Stimulus/Response Sequences

Stimulus: The User selects a root folder path and triggers the execution of the scanner.

Response: The system references the local File System, reads binary headers, and populates the library display with matches

5.1.3 Functional Requirements

REQ-1.1: The system **shall** recursively traverse all subdirectories of the target folder path.

REQ-1.2: The system **shall** parse internal data headers to identify console platforms, matching them regardless of file extension manipulation.

6. Nonfunctional Requirements

6.1. Performance

NF-1.1: «Any performance-related issue»

NF-1.«N»: «Any performance-related issue»

6.2. Security

NF-2.1: «Any security-related issue»

NF-2.«N»: «Any security-related issue»

6.«N». «Any additional “nonfunctional” blocks»